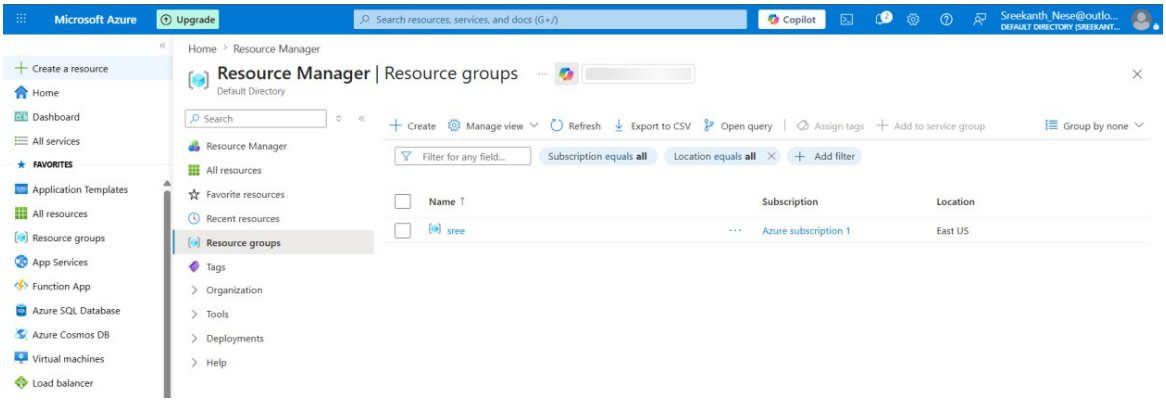
Employee & Training Cost Analytics Pipeline

Azure Blob Storage → ADF Mapping Data Flow → Azure SQL Database

This project builds an analytics pipeline that reads raw employee and training-cost CSV files from Azure Blob Storage, applies three parallel transformation branches in an Azure Data Factory Mapping Data Flow (aggregation, ranking/filtering, and derived columns), and writes the results into three separate tables in Azure SQL Database for reporting.

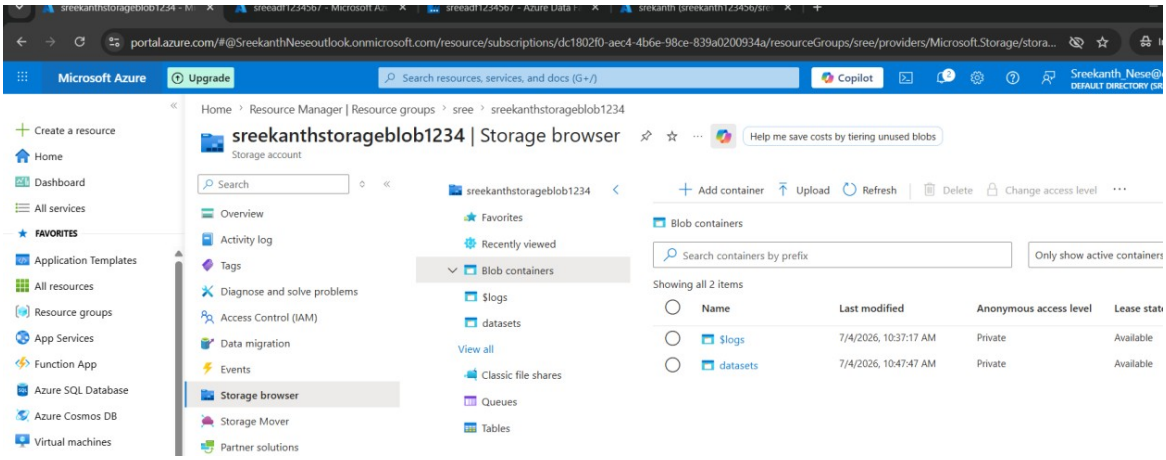
Step 1: Created a Resource Group

A Resource Group named “sree” (East US) was created to hold every resource used in this project — the storage account, Data Factory, and SQL Database — under one manageable container.

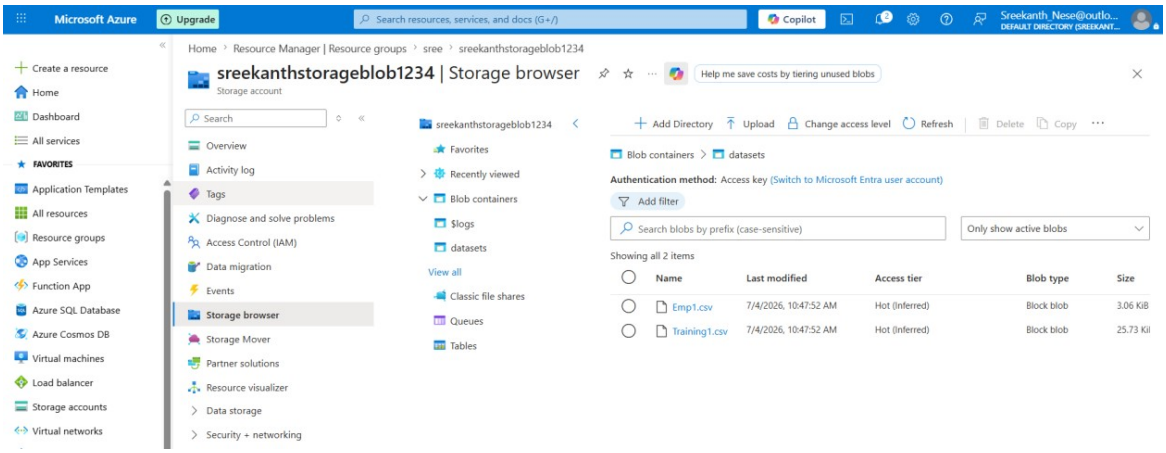


Step 2: Created a Storage Account and Uploaded Source Files

A storage account (sreekanthstorageblob1234) was created with two blob containers — blogs and datasets. The datasets container holds the raw source files for this pipeline.

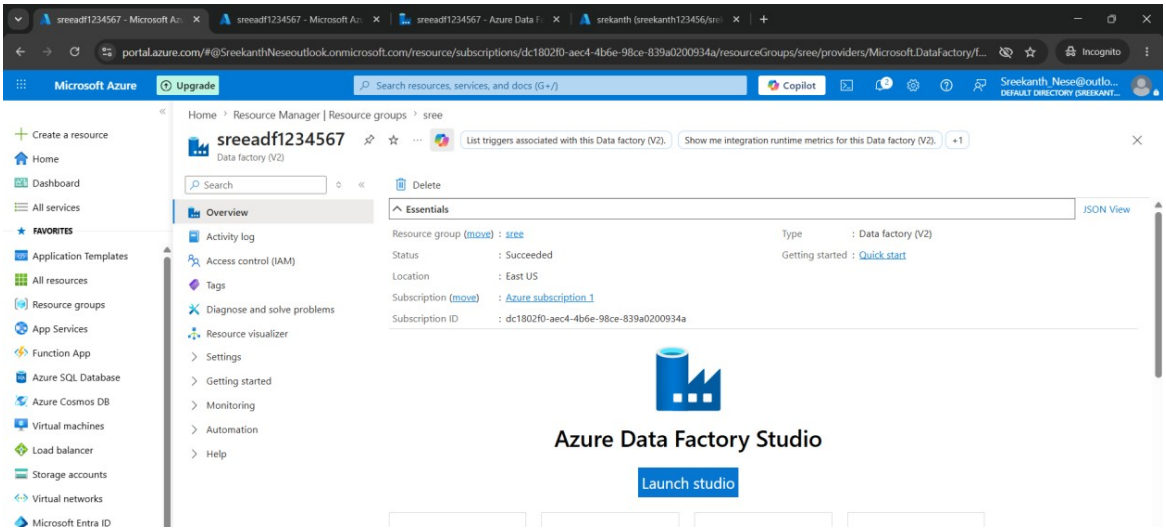


Two CSV files were uploaded into the datasets container: Emp1.csv (employee master data, ~3 KB) and Training1.csv (training/course cost data, ~25.7 KB). These are the two inputs the Data Flow reads from.



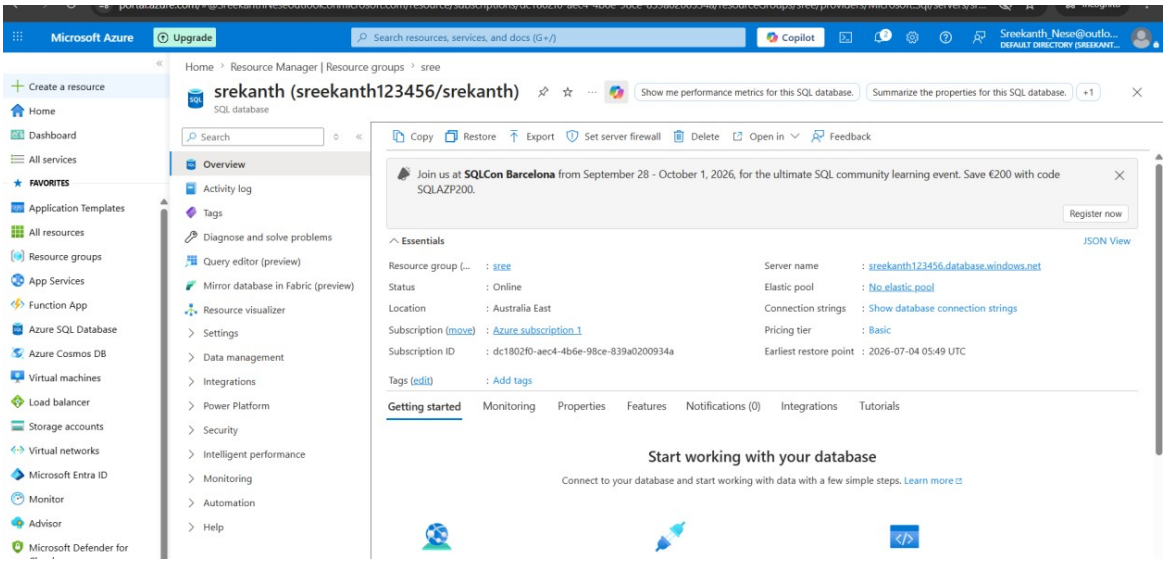
Step 3: Created the Azure Data Factory

An Azure Data Factory (sreeadf1234567) was provisioned in East US to host the pipeline, datasets, and the Mapping Data Flow used for the transformations.



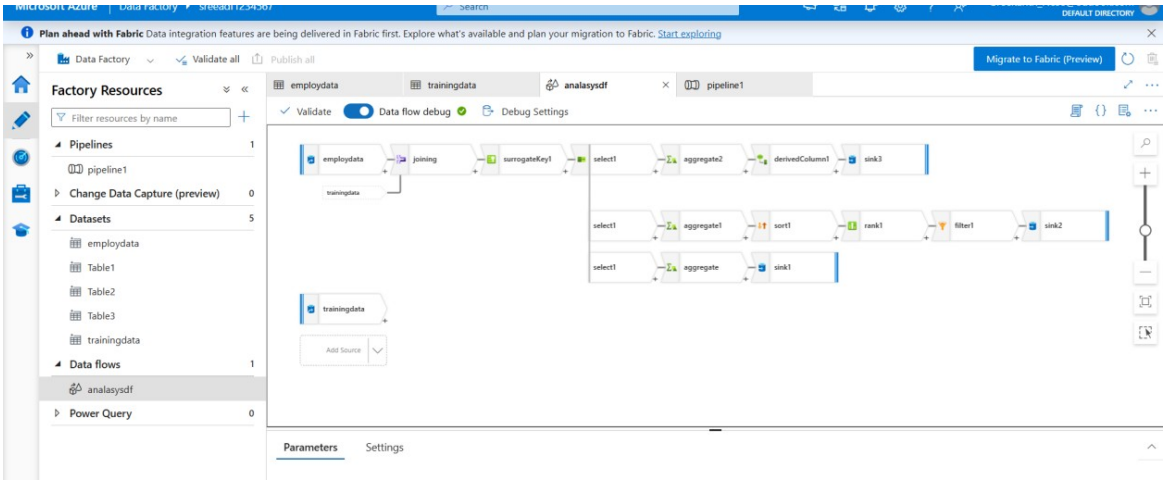
Step 4: Created the Azure SQL Database

An Azure SQL Database (srekanth, on server sreekanth123456, Australia East, Basic tier) was created as the destination for the three output tables produced by the Data Flow.



Step 5: Created Datasets and the Pipeline

Inside the Data Factory, five datasets were created: employdata and trainingdata (pointing at the two CSV files in Blob Storage), and Table1, Table2, and Table3 (pointing at the three destination tables in Azure SQL Database). A pipeline (pipeline1) was then created with a single Data Flow activity referencing the Mapping Data Flow analaysdf.



Step 6: Configured the Mapping Data Flow — Three Transformation Branches

After a shared select1 transformation (renaming the surrogate key column) reads the joined employee/training data, the flow splits into three independent branches. Each branch is broken down transformation-by-transformation below, including exactly what each step produces.

Shared Step — select1

Before the branches split, select1 runs once on the incoming joined employee + training dataset. It renames the surrogate key column (surrogateKey1) produced by the join and drops/reshapes columns so every downstream branch works from a clean, consistently-named set of fields — EmployeeID, employname, CourseCode, Gender, MaritalStatus, and Totalcost.

Branch 1 — Employee Cost → Table1

Transformation	What It Does	Output
aggregate	Groups all rows by employname and EmployeeID, and sums Totalcost for each employee across every course/training record they appear in.	One row per employee: EmployeeID, employname, and their total training cost.
sink1	Writes the aggregated per-employee rows to the destination table using the Table1 dataset, replacing/inserting rows as configured.	Table1 in Azure SQL Database, seen in the portal as EmployCost_tb — the per-employee cost table used for employee-level cost reporting.

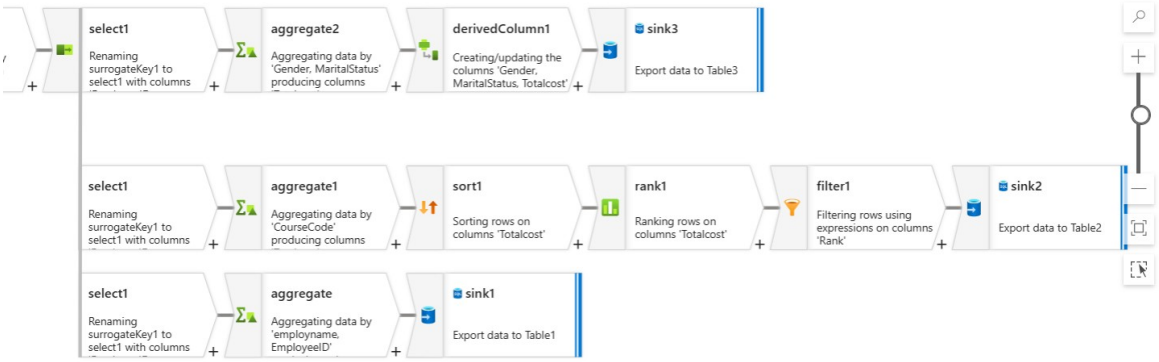
Branch 2 — Top 5 Courses by Cost → Table2

Transformation	What It Does	Output
aggregate1	Groups all rows by CourseCode and sums Totalcost, collapsing every employee's record for a course into one total-cost figure per course.	One row per course: CourseCode and its Totalcost across all enrolled employees.
sort1	Sorts the per-course rows by Totalcost in descending order, so the most expensive courses appear first.	Same rows as aggregate1, now ordered highest-to-lowest by Totalcost.
rank1	Assigns a Rank value to each row based on its position in the sorted Totalcost order. Rows with an identical Totalcost receive the same rank (e.g. two courses tied at 69300 both rank 4th).	Adds a Rank column alongside CourseCode and Totalcost, with ties sharing a rank instead of being arbitrarily split.
filter1	Applies a filter expression that keeps only rows where Rank falls within the top 5, discarding every lower-ranked course.	Just the top-ranked courses by cost (5 rows, or slightly more when a tie occurs at the cutoff).
sink2	Writes the filtered top-ranked rows to the destination table using the Table2 dataset.	Table2 in Azure SQL Database, seen in the portal as Top5 — CourseCode, Totalcost, and Rank for the costliest courses, used for cost-driver reporting.

Branch 3 — Cost by Gender & Marital Status → Table3

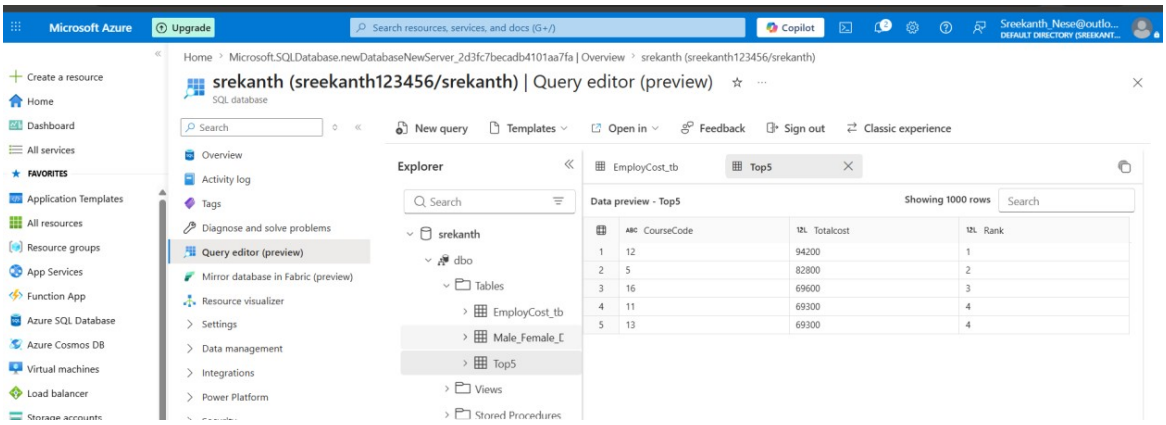
Transformation	What It Does	Output
aggregate2	Groups all rows by Gender and MaritalStatus together, summing Totalcost for each combination of the two fields.	One row per Gender + MaritalStatus combination, with the total cost for that group.
derivedColumn1	Creates/updates the Gender, MaritalStatus, and Totalcost columns — typically standardizing value formatting (e.g. consistent labels/casing) and ensuring Totalcost is the correct numeric type before it's written out.	The same grouped rows, with clean, consistently-formatted Gender, MaritalStatus, and Totalcost columns.
sink3	Writes the cleaned, grouped rows to the	Table3 in Azure SQL Database, seen in

Transformation	What It Does	Output
	destination table using the Table3 dataset.	the portal as Male_Female_tb — cost broken down by gender and marital status, used for demographic cost-split reporting.



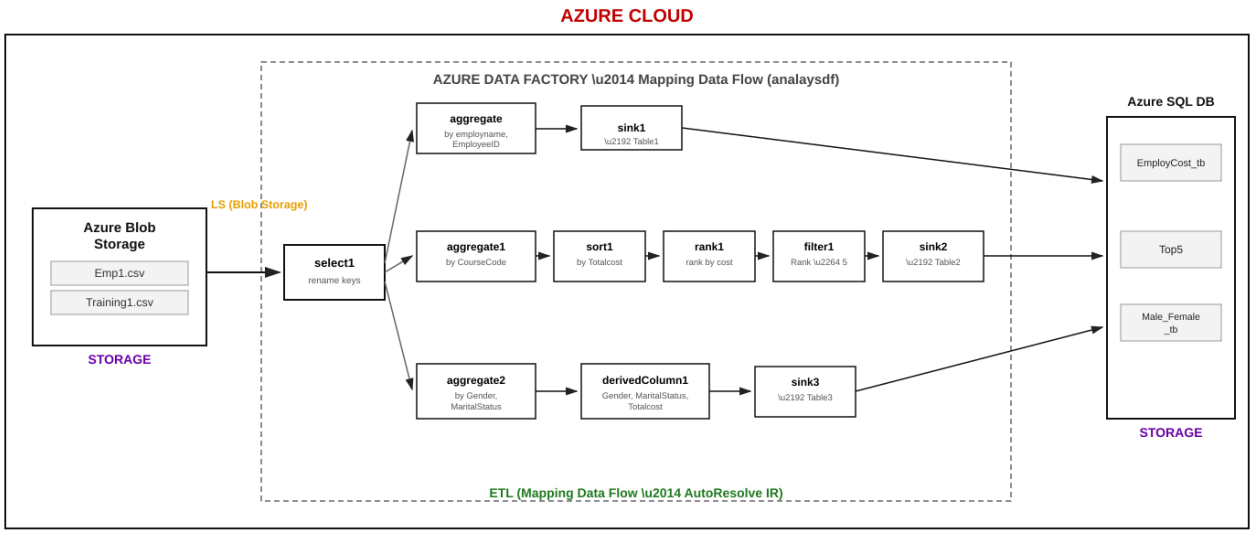
Step 7: Validated the Results in Azure SQL Database

The three output tables — EmployCost_tb, Male_Female_tb, and Top5 — were verified using the Azure SQL Query Editor. The Top5 table preview below shows courses ranked by Totalcost, with matching cost values correctly sharing the same rank (e.g. CourseCode 11 and 13 both ranked 4th at a Totalcost of 69300) — confirming the aggregate → sort → rank → filter branch worked as designed.



Architecture Diagram

The diagram below summarizes the full pipeline: Blob Storage holds the raw CSV inputs, the ADF Mapping Data Flow applies a shared select transformation and then fans out into the three aggregation branches described above, and each branch's sink writes into its own table in Azure SQL Database.



Key Takeaway

Structuring the transformation as a single Data Flow with a shared source and three parallel branches avoided re-reading or re-joining the source data three times. Each branch focuses on one specific business question — cost per employee, top 5 courses by cost, and cost split by gender/marital status — and writes straight to its own table, keeping the pipeline compact while still producing three distinct, reporting-ready outputs from a single run.